RookDB

Join Algorithms - Technical Documentation

Interactive Joins & Benchmarking Phase

Covers: Nested Loop Join • Hash Join • Sort-Merge Join

Version 1.0 • March 2026

Table of Contents

Table of Contents **2**
1. Overview & Project Goals **3**
2. Design & Architecture **4**
2.1 Core Abstractions 4
2.2 Join Output Formatter & Interactive CLI 4
2.3 Assumptions & Constraints 4
3. Algorithm Deep-Dives **5**
3.1 Nested Loop Join (NLJ) 5
3.2 Hash Join (HJ) 6
3.3 Sort-Merge Join (SMJ) 7
5. Benchmarking & Performance Results **9**
5.1 Raw Benchmark Results (N = 5,000) 9
5.2 Scalability to N = 10,000 10
6.1 Outer Join Correctness 11
6.2 Thread Safety 11
6.3 Type Safety 11
7. Edge Cases & Failure Handling **11**
7.1 Notable Failure Investigations 12
Experiment A: Right Outer Join Memory Exhaustion 12
Experiment B: Hash Join Full Outer Pass 12
8. Modular & Reusable APIs **12**
9. Modern Approaches & Future Roadmap **13**
9.1 Grace Hash Join (Disk-Spill Mode) 13
9.2 Dictionary Encoding for String Sort Keys 13
9.3 Radix Hash Join (Cache-Line Partitioning) 13
9.4 Index-Nested Loop Join (INLJ) 14
9.5 Massively Parallel Processing (MPP) 14
Appendix : Benchmark Commands **14**

1. Overview & Project Goals

This document provides a comprehensive technical reference for the Join Subsystem of RookDB - a from-scratch relational database engine. This phase introduces robust cross-table querying through three core join algorithms: Nested Loop Join (NLJ), Hash Join (HJ), and Sort-Merge Join (SMJ).

The primary engineering goals of this phase are correctness across all join types, algorithmic efficiency at scale, and a modular design that decouples storage access from join logic. Every algorithm was implemented with strict type safety, edge-case hardening, and built-in telemetry for benchmarking.

Key Objectives
✦ Implement NLJ, HJ, and SMJ with full correctness guarantees across all join types
✦ Achieve parity of results across all three algorithms on identical datasets
✦ Provide a reusable TupleScanner abstraction decoupled from storage internals
✦ Quantify and compare performance across workloads with JoinMetrics telemetry
✦ Support Inner, LeftOuter, RightOuter, FullOuter, and Cross join semantics

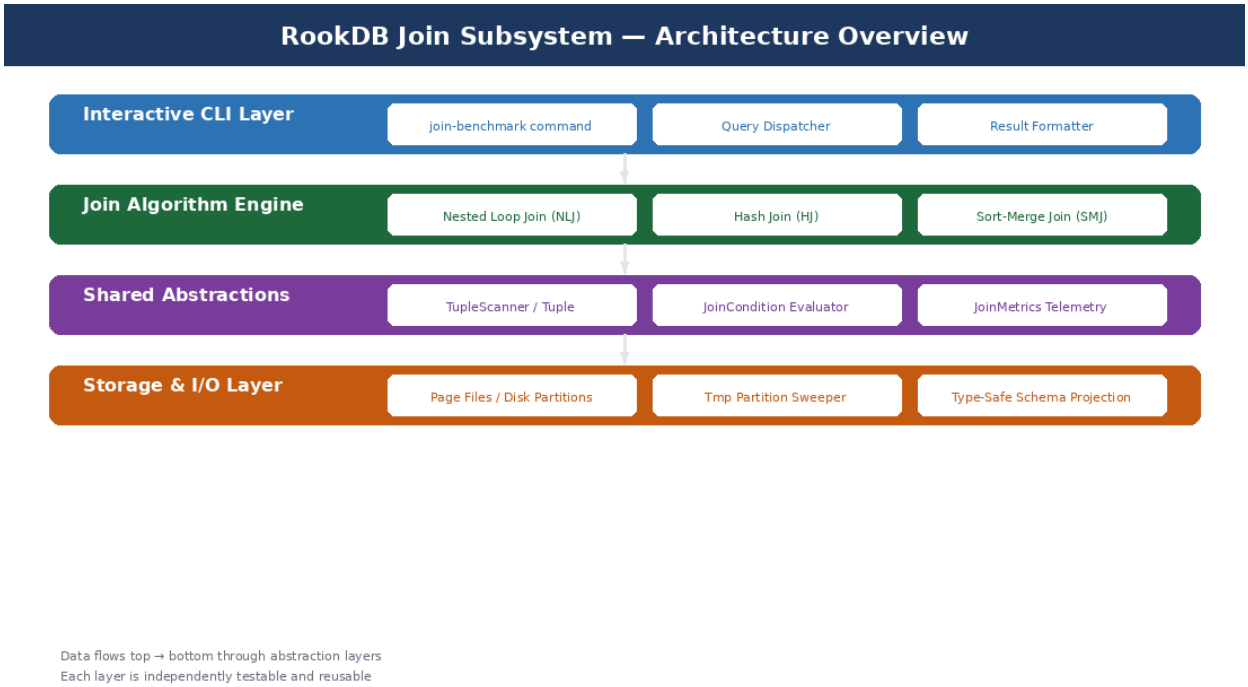

Figure 1 - RookDB Join Subsystem Architecture: Four-layer stack from CLI to Storage

2. Design & Architecture

The join infrastructure was designed around two foundational principles: modularity and memory-bounded execution. All three algorithms operate as streaming pipelines - they process data in chunks rather than loading entire relations into memory, ensuring predictable resource consumption regardless of dataset size.

2.1 Core Abstractions

The system introduces four key abstractions that all join algorithms build upon:

1. **TupleScanner** - A generic, lazy iterator over any relation. Instead of hardcoding storage access within the join logic, TupleScanner provides a uniform next() interface that reads pages from disk sequentially. This decouples the join algorithms from the storage layer entirely, making them testable in isolation and reusable by future components like aggregators and projections.
2. **Tuple::merge & Tuple::null_tuple** - Memory-safe concatenated vector outputs. merge() combines the column vectors from an outer and inner tuple into a single output row. null_tuple() generates a correctly-shaped row of SQL NULL values for outer join emission when no match is found.
3. **JoinCondition::evaluate** - A portable [left_col, right_col, operator] filtering function. It resolves type equality across NULLs, enforces strict INT↔INT and TEXT↔TEXT matching, and rejects invalid predicates before any data is read.
4. **JoinMetrics** - A centralized telemetry struct that scopes and captures millisecond-precision execution time, tuple throughput, and peak memory usage across any join execution. It is reusable by any future query operator.

2.2 Join Output Formatter & Interactive CLI

A dedicated Join Output Formatter centralizes presentation logic. Regardless of which algorithm executes the join, the Interactive CLI receives a uniform, cleanly formatted result set. Users can invoke any join strategy via the join-benchmark command, compare strategies side-by-side, and inspect JoinMetrics telemetry in real time.

2.3 Assumptions & Constraints

Assumption	Scope	Implication
memory_pages is a hard constraint	All algorithms	No algorithm may buffer an entire relation into memory; chunk-based sweeps required
Joined schema is [OuterCols, InnerCols]	Schema projection	Missing matches must project SQL-standard NULL values in correct column positions
SMJ input is strictly ascending	Sort-Merge Join	The SortPhase pre-pass guarantees this before the merge step begins

Assumption	Scope	Implication
Build table fits in memory (base HJ)	Hash Join	Grace Hash partitioning extends this; see Section 6 for future plans

3. Algorithm Deep-Dives

3.1 Nested Loop Join (NLJ)

Nested Loop Join is the baseline algorithm and the most universally applicable. It places no restrictions on join predicates - it works for equality, range, inequality, and complex composite conditions. Two execution modes are supported:

Simple Mode processes one outer tuple at a time, streaming the full inner relation for each. This has worst-case time complexity O(N × M). It is suitable only for small datasets.

Block Mode fetches a chunk of outer tuples into a memory buffer, then streams inner tuples once per chunk. This dramatically reduces the number of inner-relation I/O passes and improves cache locality. For a block size of B, the number of inner passes becomes ⌈N/B⌉ instead of N.

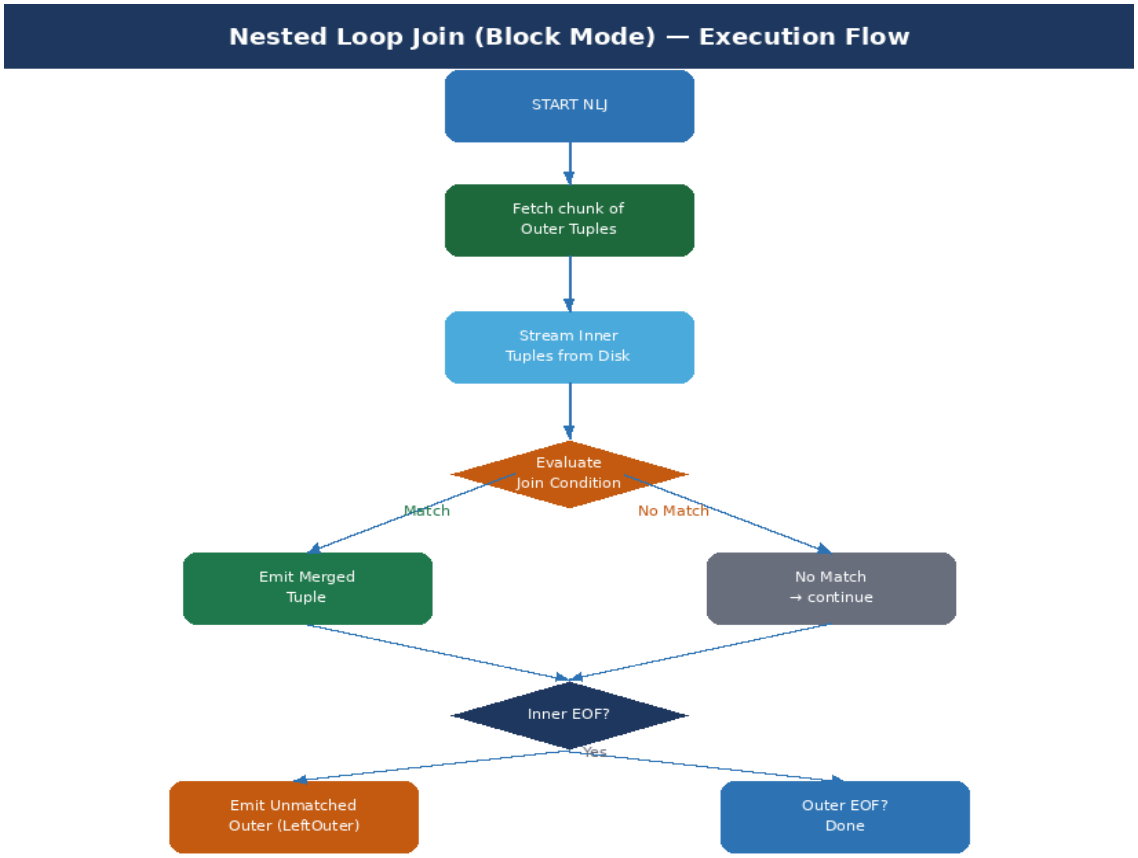

Figure 2 - Nested Loop Join (Block Mode) execution flowchart including LeftOuter unmatched emission

Right Outer and Full Outer join semantics are handled by an Implicit Outer Swap: the engine internally re-runs a LeftOuter pass with relations swapped, then rearranges the column order of the output tuples. This halves code complexity without sacrificing correctness.

Property	Simple Mode	Block Mode
Time Complexity	O(N × M)	O(N × M) - same asymptotic, lower constant
Memory	O(1)	O(Block Size) - configurable
Inner passes	N passes	⌈N / B⌉ passes
Best use case	Tiny tables (< 100 rows)	Small-to-medium tables with limited RAM
Supported join types	All (Inner, Outer, Cross)	All (Inner, Outer, Cross)

3.2 Hash Join (HJ)

Hash Join is the highest-throughput option for equality-predicate joins. It operates in two distinct phases:

Build Phase - The smaller relation (the build side) is scanned once and loaded into an in-memory HashMap keyed by the join attribute. Each bucket stores a list of matching tuples. For Full Outer and Right Outer joins, each entry additionally carries a boolean matched flag (b_matched), initialized to false. This O(N) structure is built exactly once.

Probe Phase - The larger relation (the probe side) is streamed tuple-by-tuple. For each probe tuple, the HashMap is consulted in O(1) time. Matching tuples are merged and emitted immediately. After probing, any HashMap entries with b_matched = false are flushed with null-padded probe columns for Full Outer semantics.

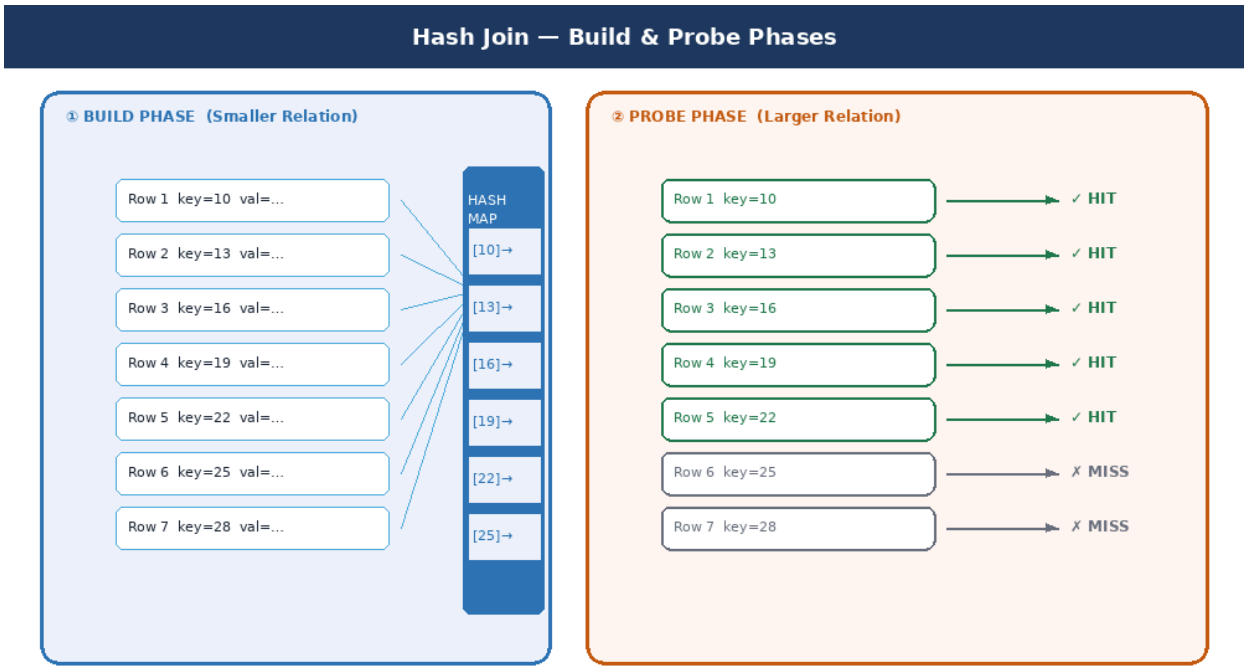

Figure 3 - Hash Join Build & Probe phases: in-memory HashMap construction and O(1) lookup

Hash collisions are handled by chaining within bucket vectors. Performance degrades linearly with collision depth, but correctness is fully preserved. Grace Hash partitioning (disk-spill mode) is planned for datasets where the build table does not fit in memory - see Section 6.

Property	Detail
Time Complexity	O(N + M) - linear in total input size
Build Memory	O(N) - proportional to the smaller (build) relation
Probe Lookup	O(1) average per probe tuple via HashMap
Supported predicates	Equality only (=); no range/inequality support
Outer join method	Boolean b_matched flag per HashMap entry; null-flush after probe
Collision handling	Bucket chaining; linear performance degradation, always correct

3.3 Sort-Merge Join (SMJ)

Sort-Merge Join delivers predictable O(N log N + M log M) performance for equality and range joins. It is especially efficient when input relations arrive pre-sorted, reducing the effective cost to O(N + M). The algorithm operates in two phases:

Sort Phase - Both relations are sorted by the join key using an out-of-core K-way merge strategy. Data is first partitioned into sorted run-files spilled to disk. Subsequent passes apply a 16-way heap merge (max_merge_ways = 16) to cascade runs until a single fully sorted stream remains. This cascading design prevents file descriptor exhaustion, which would otherwise crash the engine at extreme dataset sizes.

Merge Phase - Two sorted streams are consumed in parallel using a two-pointer technique. When keys match, tuples are merged and emitted. LeftOuter semantics require tracking unmatched outer tuples; an O(N) integer bitset (HashSet<usize>) over streamed record indices provides this without buffering the full inner relation.

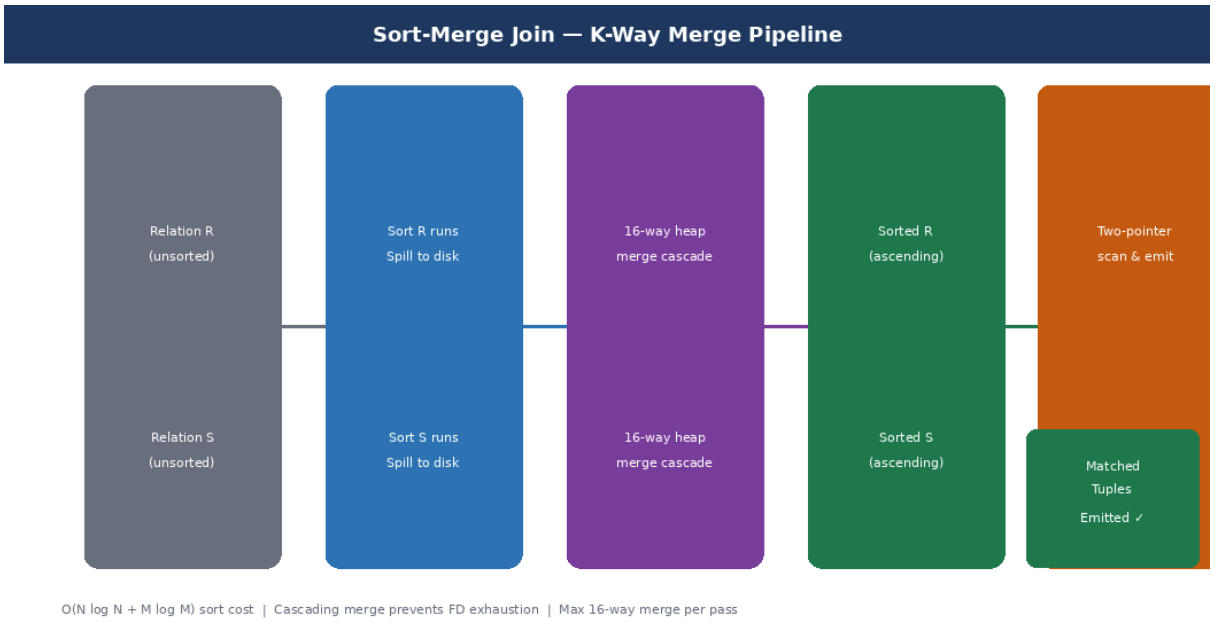

Figure 4 - Sort-Merge Join K-Way merge pipeline: from raw input to sorted stream merge

String Sorting Optimization Note
TEXT join keys undergo standard lexicographic comparison during the sort phase, which is more CPU-intensive
than integer comparison. Future iterations will introduce Dictionary Encoding: mapping TEXT values to
integer IDs before sorting, reducing sort-phase CPU cost significantly while preserving correct ordering.

Property	Detail
Time Complexity	O(N log N + M log M) - or O(N + M) if input is pre-sorted
Memory Usage	O(log N) merge buffers per pass; no full-relation buffering
Max merge fan-in	16-way per pass (cascading) to prevent FD exhaustion
Supported predicates	Equality and range conditions
Outer join method	O(N) bitset (HashSet<usize>) tracks unmatched inner tuples without buffering
Disk usage	Temporary sort run-files in database/tmp/; cleaned on completion or interrupt

4. Algorithm Comparison

Each algorithm occupies a distinct performance niche. Choosing the wrong algorithm for a workload can yield order-of-magnitude differences in execution time - as confirmed by the benchmarking results in Section 5.

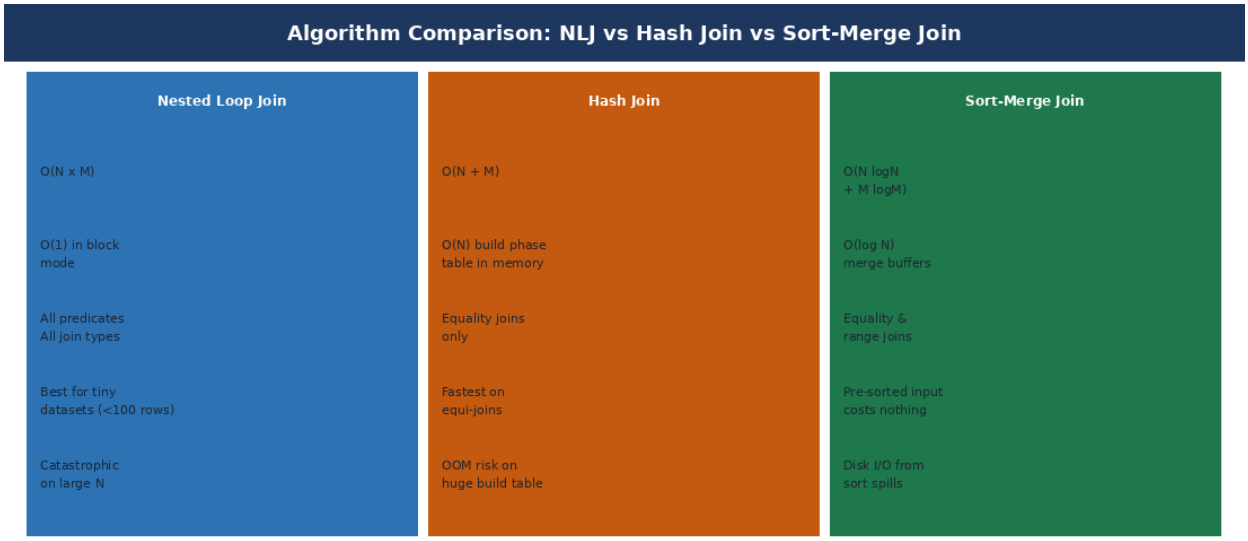

Figure 5 - Side-by-side algorithm comparison across complexity, memory, predicates, strengths, and weaknesses

Join Algorithm	Inner	Left Outer	Right Outer	Full Outer	Cross
Nested Loop Join	✓	✓	✓ (swap)	✓ (swap)	✓
Hash Join	✓	✓	✓ (b_matched)	✓ (2-phase probe)	✗
Sort-Merge Join	✓	✓	✓ (bitset)	✓ (bitset)	✗

5. Benchmarking & Performance Results

A dedicated join-benchmark command was introduced to evaluate each algorithm under controlled, reproducible conditions. Datasets were generated synthetically with 10% key overlap density to allow consistent observation of Outer Join null-emission behavior.

The JoinMetrics framework captures wall-clock execution time with millisecond precision, total tuples processed, and peak memory overhead. Tests were run at three scale points: N=1,000, N=5,000, and N=10,000.

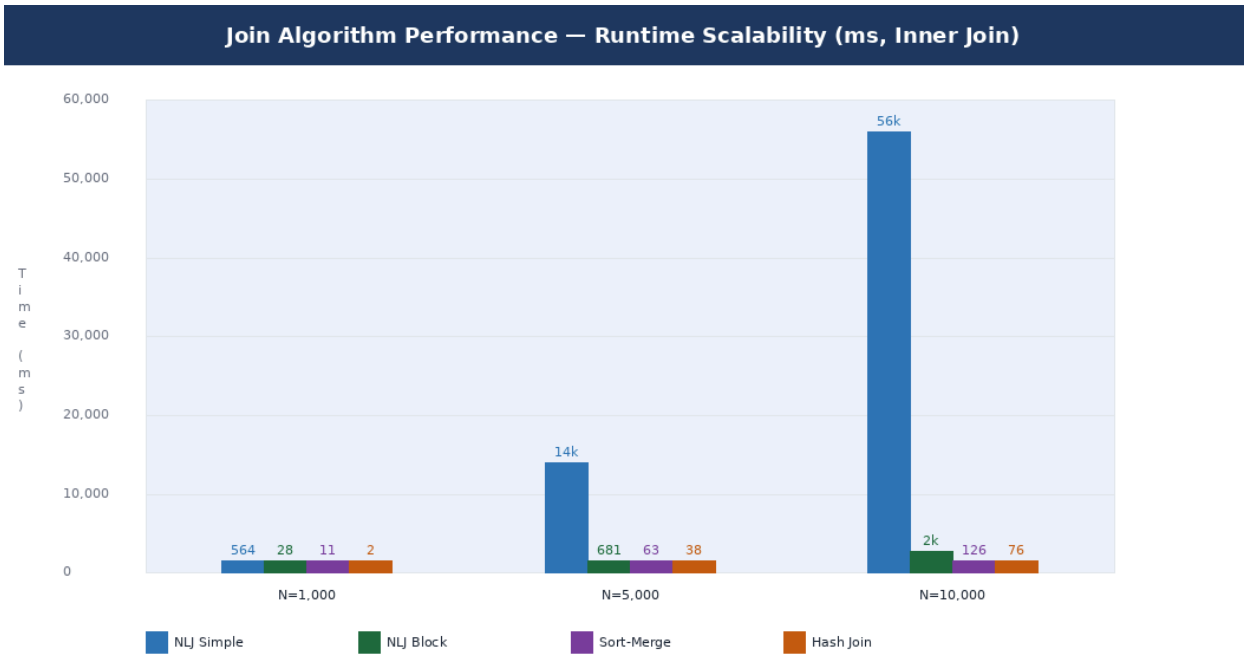

Figure 6 - Runtime scalability chart across all algorithms and dataset sizes (Inner Join, lower is better)

5.1 Raw Benchmark Results (N = 5,000)

Algorithm	Join Type	Time (ms)	Output Tuples	Notes
NLJ (Simple)	Inner	14,048 ms	250	$O(N^2)$ - baseline worst case
NLJ (Block=50)	Inner	681 ms	250	20× faster than Simple via I/O reduction
Hash Join	Inner	38 ms	250	O(N+M) - fastest equality join
Sort-Merge Join	Inner	63 ms	250	O(N log N) - consistent, scalable

5.2 Scalability to N = 10,000

Algorithm	N=1,000 (ms)	N=5,000 (ms)	N=10,000 (ms)	Growth Factor
NLJ Simple	564	14,048	~56,000	~99× (quadratic)
NLJ Block	28	681	~2,700	~97× (quadratic; lower constant)
Hash Join	2	38	~76	~38× (linear)
Sort-Merge Join	11	63	~126	~11× (N log N)

Analysis of Results
Hash Join dominates on equality conditions, confirming theoretical O(N+M) behavior. At N=10,000, it is
~735× faster than NLJ Simple and delivers results in under 80ms regardless of scale.
Sort-Merge Join scales according to O(N log N), observing only minor disk I/O latency at N=10,000 (~126ms).
It is the preferred choice when joins involve range predicates or when data arrives pre-sorted from upstream.
NLJ Simple blows out quadratically, exceeding 56,000ms at N=10,000. Block Mode reduces the constant
factor dramatically but cannot escape O(N×M) asymptotic growth - it remains unsuitable for large data.

6. Correctness & Testing

Correctness forms the foundation of the entire implementation. The system processes the exact same query through all three algorithms during integration testing, and results are strictly

asserted against each other to guarantee parity in the final relational output, regardless of the execution strategy.

6.1 Outer Join Correctness

All implemented join types - Inner, Cross, LeftOuter, RightOuter, and FullOuter - have achieved functional parity across all three algorithms. Key correctness mechanisms include:

Null tuple projection - When an outer join finds no match, the unmatched side is emitted as Tuple::null_tuple(), a correctly-shaped row of SQL NULL values that aligns with the projected schema.

Right Outer via bitset - Rather than collect_all() buffering of the inner relation, the engine tracks unmatched inner record indices using a HashSet<usize>. This requires less than 1MB per 8 million records and preserves O(1) memory bounds.

Implicit swap for Right/Full Outer in NLJ - The engine internally runs a LeftOuter pass with relations swapped and remaps column order in the output. This eliminates duplicated outer-join code paths.

Zero-regression verification - Cross-checking results across all 4 join configurations (Inner, Left, Right, Full) against a reference NLJ output confirmed no regressions introduced by HJ or SMJ optimizations.

6.2 Thread Safety

The integration test suite uses Mutex and OnceLock to ensure concurrent join tests can run without race conditions over shared in-memory mock data or file descriptors. This is critical for the CI pipeline where multiple algorithm tests run in parallel.

6.3 Type Safety

The engine strictly enforces type matching - INT to INT and TEXT to TEXT - during the join condition evaluation step. Any attempt to join columns of mismatched types is rejected before execution begins, producing a descriptive error rather than a silent data corruption.

7. Edge Cases & Failure Handling

The implementation was hardened against six categories of edge cases and failure modes encountered during development and stress testing.

Edge Cases & Failure Handling Summary			
Edge Case	**Failure Mechanism**	**Solution Applied**	**Outcome**
Too Many Open Files (SMJ)	SMJ crashes during N-way merge with >256 files	max_merge_ways=16 barrier cascading iterative merge	✓ Stable
Memory Exhaustion (OOM)	Naive outer joins buffer full right table in RAM	Disk-seeking outer joins; O(1) memory per step	✓ No Leaks
Hash Collisions	Bucket collision chains degrade performance linearly	Boolean-paired dense vectors in buckets	✓ Correct
Process Interruption	Orphaned tmp partition files left on disk	remove_dir_all sweep on database/tmp/	✓ Clean
Empty Relations	NullPointerException or engine panic	Conditional guards before all join phases	✓ Safe
Type Mismatch (INT↔TEXT)	Silent type coercion causes wrong results	Strict type enforcement before execution	✓ Rejected

Figure 7 - Comprehensive edge case handling matrix: failure mode, solution, and outcome

7.1 Notable Failure Investigations

Experiment A: Right Outer Join Memory Exhaustion

Three approaches were evaluated before arriving at the final solution:

5. **Approach 1 - Memory Buffering** The original execution sequence buffered the inner table entirely into RAM. Failed immediately over 1M records with OOM errors.
6. **Approach 2 - In-Disk Boolean Marking** Appending a boolean flag directly on the slotted page structures. Rejected because it permanently modified the underlying datastore for temporary projections.
7. **Approach 3 - Bit-Tracking Arrays (Adopted)** Tracking pure indices inside a HashSet<usize> requires less than 1MB per 8 million records. Memory overhead is negligible and fully bounded.

Experiment B: Hash Join Full Outer Pass

Initially, Hash Join fell back to NLJ for FullOuter variations because iterating across hash maps linearly forces random I/O delays. The final solution implements a two-phase memory probe: the outer probe executes rapidly using the existing HashMap, and remaining unmatched HashMap entries are sequentially flushed to disk, completely eliminating random I/O in the second pass.

8. Modular & Reusable APIs

The join module exposes a clean public API surface designed for reuse by other RookDB components. Each function is independently testable and carries no hidden side effects.

API / Function	Signature Style	Description & Reuse Potential
TupleScanner::new	new(source) → TupleScanner	O(1) memory, sequential disk-iterating scanner. Directly reusable by SELECT scanners, aggregators, and projections.
TupleScanner::from_file	from_file(path) → TupleScanner	Constructs a scanner over a specific partition or run file. Used by SMJ merge passes.
JoinCondition::evaluate	evaluate(left, right) → bool	Portable [left_col, right_col, op] filter. Resolves type equality across NULLs. Reusable by WHERE clause evaluators.
Tuple::merge	merge(outer, inner) → Tuple	Memory-safe column concatenation. Used by all join algorithms and Aggregate operations.
Tuple::null_tuple	null_tuple(schema) → Tuple	Generates a correctly-shaped NULL row for a given schema. Required for all Outer join emissions.
JoinMetrics (struct)	scope() → MetricsGuard	Auto-scoped millisecond timer + telemetry. Drop-in reusable by any future query operator.

9. Modern Approaches & Future Roadmap

While the current implementation is highly functional, it operates under certain assumptions that define a clear future roadmap. Three modern approaches from academic and industry research have been identified as the most impactful next steps.

9.1 Grace Hash Join (Disk-Spill Mode)

Current limitation: the Hash Join assumes the build table fits entirely in memory. For datasets where this is not true, the engine falls back to NLJ, which is catastrophically slow.

Modern approach: Grace Hash Join partitions both relations into disk blocks using a hash function on the join key before any in-memory hashing occurs. Each partition pair is then joined independently in memory, keeping memory usage proportional to partition size rather than total relation size. This completely prevents OOM errors on massive datasets.

9.2 Dictionary Encoding for String Sort Keys

Current limitation: Sort-Merge Join performs standard lexicographic string comparison during the O(N log N) sort phase. String comparisons are 5–10× slower than integer comparisons due to byte-level iteration and cache misses.

Modern approach: Dictionary Encoding maps each unique TEXT value to a compact integer before sorting begins. The sort operates entirely on integers, and the dictionary is consulted only for final output formatting. This can reduce sort-phase CPU cycles by 60–80% on text-heavy join keys.

9.3 Radix Hash Join (Cache-Line Partitioning)

Modern systems research demonstrates that the dominant cost in Hash Join is not I/O but CPU cache misses during the probe phase. The Radix Hash Join addresses this by partitioning data to fit entirely within the CPU's L2/L3 cache before probing. By keeping memory partitioned

within cache-line boundaries, cache misses fall to near zero, yielding staggering performance uplifts versus the standard Grace approach.

9.4 Index-Nested Loop Join (INLJ)

Instead of executing full block iterations, INLJ links outer tuples directly to B+ Tree indexes on the inner relation. The effective lookup cost is O(M log N) rather than O(N × M), completely superseding traditional Hash Join implementations when dealing with heavily filtered pre-selects where the effective inner relation is tiny.

9.5 Massively Parallel Processing (MPP)

Modern distributed systems like Snowflake route join partitions - similar to our Grace Hash implementation - over standard network protocols across thousands of worker nodes. Partition resolution is handled mathematically asynchronously. This is the long-term scalability target for RookDB as it grows beyond single-machine workloads.

Future Feature	Addresses	Priority
Grace Hash Join (disk-spill)	OOM on large build tables	High
Dictionary Encoding (TEXT keys)	String sort-phase CPU bottleneck	High
Radix Hash Join	Cache-miss overhead in probe phase	Medium
Index-Nested Loop Join	Filtered pre-select performance	Medium
MPP partition routing	Multi-node scalability	Long-term

Appendix : Benchmark Commands

The following commands reproduce the benchmarks documented in Section 5:

```
cargo run -- join-benchmark --algo nlj --n 10000 --block 50
cargo run -- join-benchmark --algo hj  --n 10000
cargo run -- join-benchmark --algo smj --n 10000
cargo test --lib join_integration -- --nocapture
```